

PRA-vLLM: Separating Query-Addressed Logical Memory from Paged KV Placement

Jorge Simão
EInnovator R&D

August 2026

Abstract

vLLM virtualizes where sequential key-value (KV) blocks reside. Progressive Retrieval Attention (PRA) virtualizes a different choice: which subset of a logically large, non-prefix memory is active for a query. We define a capability-honest integration boundary between these mappings, implement a stable logical PRA block namespace and residency state machine, and implement a vLLM-Metal store that places selected K/V in native paged-cache blocks. On an Apple M5 with vLLM 0.28.0 and the vLLM-Metal plugin, a 25-request smoke study separates exact-prefix reuse from selected-memory recovery. A selected 365-token prompt and the 1,577-token full context both recover the codeword in 5/5 requests; the selected condition uses 76.9% fewer visible tokens and reduces cold time to first token (TTFT) from 776.6 to 80.7 ms. Warm TTFT is 46.2 versus 52.9 ms, while vLLM reports an aggregate prefix-cache hit rate of 75.3–83.4%. These measurements establish a strong black-box baseline. Separately, 80 controlled kernel measurements over five seeds, 16–512 selected tokens, and concurrency 1–8 match dense attention within maximum absolute error 0.00391. Warm native paged attention rises from 0.489 to 0.642 ms; immutable block sharing avoids up to 14.0 MB at concurrency eight. A subsequent five-seed live V1 experiment reserves scheduler-invisible tail pages and prepends them only to attention block tables. Native and full context recover 5/5 answers, disabled memory recovers 0/5, cleanup leaks are 0/5, and wrong memory controls follow the wrong fact in 5/5 cases. This closes controlled V1 generation on Metal. A second five-seed cohort combines native PRA with APC over concurrency 1–8: selected memory recovers 75/75 answers, alternating selected and ordinary requests score 40/40 on their expected outcome, wrong memory redirects 30/40 outputs, and throughput reaches 34.41 requests/s at concurrency eight. A selection-derived APC salt prevents the same visible query from reusing state created under different hidden memory; after the first selected request, APC supplies 128–144 prompt tokens. Connectors, online server tails, cross-model replication, and CUDA integration remain open. A final matched-selection study uses the same routed QASPER, HotpotQA, and 2WikiMultiHopQA evidence for visible E0 and native E2 execution. E2 removes 90.6–93.0% of visible prompt tokens, but exact output parity is only 5.0–20.0% and mean F1 falls on HotpotQA and 2Wiki. This isolates an unresolved native transport gap rather than a routing error, so matched-quality promotion remains open.

1 Introduction

Long-running LLM services face two related but distinct memory problems. Sequential request KV must be allocated, paged, reused, and evicted. Separately, a retained corpus or session history need not participate in every query. The first problem is physical. The second is logical.

PagedAttention and vLLM address the physical problem by decoupling logical sequence positions from non-contiguous cache blocks and scheduling requests over the resulting pool [1]. PRA addresses the logical problem by separating address state from detail state and activating only selected non-prefix memory. The combined mapping is

logical PRA memory $\rightarrow$ query-selected logical blocks $\rightarrow$ physical engine blocks.

The middle arrow cannot be inferred from ordinary prefix equality. Two requests may share the same resource but select different intervals; two physical cache slots may hold the same immutable detail while using request-local position metadata.

This paper asks whether exposing that middle mapping to vLLM improves the quality-adjusted memory, latency, and throughput frontier beyond an optimized route-then-materialize baseline. The current iteration makes three contributions:

1. a stable block identity and validated residency lifecycle independent of vLLM’s physical block numbers;
2. a vLLM adapter whose advertised PRA level is derived from an installed native executor, with tenant-scoped salt support at E0 and hidden-memory cache identity at E2;
3. native Metal paged-K/V kernel and live V1 generation studies, alongside the selected-text serving baseline, with parity, causality, cleanup, latency, and physical-sharing controls.

2 Two Independent Reuse Systems

Automatic Prefix Caching (APC) reuses an exact token prefix. PRA selects a non-prefix logical resource or interval. These mechanisms can coexist:

$$\underbrace{\text{stable conversation prefix}}_{\text{APC}} + \underbrace{\text{query-selected retained memory}}_{\text{PRA}}.$$

The security scopes are also independent. Prefix cache presence does not grant resource authorization, and a PRA block selected for one tenant cannot be reused by another merely because its physical bytes remain resident.

There is a less obvious correctness requirement. vLLM’s APC key ordinarily depends on visible prompt tokens, whereas native PRA memory is absent from that prompt. Two requests with the same query but different selected memories would therefore collide. The bridge derives an opaque cache salt from the ordered logical selection, selected-token count, position base, consumer layers, and an optional deployment secret. Equal selection and geometry preserve warm reuse; any change creates a distinct namespace. The digest exposes neither resource names nor the secret. This protects correctness at the cost of losing APC sharing across different hidden-memory selections; reuse within one stable selection remains available.

Our E0/G10 baseline performs selection before the engine, injects the selected text into the current turn, and lets ordinary vLLM execute the request. This is the baseline a deeper integration must beat. Paged KV being active inside vLLM does not by itself make this path native PRA.

3 Logical PRA Block Contract

The implementation in `src/prahf/engine_memory.py` defines identity with the following fields:

Field group	Meaning
scope	tenant, optional session, and authorization scope
resource	resource identifier, immutable version, and record type
interval	half-open source-token interval $[i, j)$
model	immutable model revision, consumer layer, dtype, and K/V layout
policy	materialization profile and source-position/rebinding policy

The canonical JSON representation is hashed to produce an engine-safe logical key. Physical vLLM block numbers are deliberately absent. A resource update invalidates every matching physical realization without changing the identity of unrelated versions.

The state machine is

$$\begin{aligned} \text{INDEXEDONLY} &\rightarrow \text{OFFDEVICE} \rightarrow \text{PREFETCHING} \rightarrow \text{RESIDENT}, \\ \text{RESIDENT} &\leftrightarrow \text{PINNED}, \quad \text{RESIDENT} \rightarrow \text{EVICTABLE} \rightarrow \text{OFFDEVICE}, \end{aligned}$$

with an explicit terminal INVALID state. A transition to resident or pinned is rejected unless the engine bridge supplies at least one physical handle. Selection checks tenant and authorization scope before incrementing reuse counters. Snapshots report address bytes, logical detail bytes, resident detail bytes, indexed-only detail bytes, selections, and reuses separately.

4 Capability-Honest vLLM Adapter

The adapter in `src/pravllm/adapter.py` has two forms. Without a native executor it is an E0 OpenAI-compatible facade. It advertises APC and text fallback, but *not* logical references or native K/V. A deployment secret and tenant identifier derive a SHA-256 `cache_salt`; neither value is placed directly in the request.

With a native executor, the adapter becomes E2 and delegates generation, streaming, session close, and block-store access to that executor. This dependency-injection boundary is intentional. Metadata registration alone cannot promote the capability matrix. An executor must map selected identities to physical blocks and preserve K/V layout, GQA/MQA, masks, positions, consumer layers, request lifetime, and one correct attention normalization.

The concrete bridge is implemented in the vLLM-Metal native module. It maps each immutable logical key to ordinary native paged-cache pages and packs post-position K/V in $[\text{block}, \text{token}, H_{kv}, D]$ layout, builds one block table per request, and invokes the native Metal `paged_attention_primitive`. Multiple requests may reference one physical handle. This is the verified attention-kernel foundation used by the live V1 bridge below.

The next integration layer is implemented in `src/pravllm/v1_metadata.py`. A request registers immutable logical keys, selected block tables per cache group, selected-token count, source-position base, and consumer layers. At each V1 step the registry combines that object with scheduler-owned local cache starts and query-token counts without rewriting either. Thus two quantities remain explicit:

$$p_{q,0} = p_{\text{source}} + T_{\text{local}}, \quad T_{\text{attn}} = T_{\text{selected}} + T_{\text{local}} + T_q.$$

Appending selected pages to ordinary scheduler block tables would instead make V1 count them as sequential prefix state and assign the wrong query positions. The registry validates immutable request registration and explicit cleanup. `src/pravllm/v1_native.py` supplies the corresponding model-runner hook: it extends the physical Metal cache with a reserved tail, writes immutable selected pages there, and prepends those page IDs only to request-local attention metadata. Scheduler slot mappings, sequential token counts, and ordinary prefix identities remain unchanged. Query positions advance by the source-position base. This first E2 path requires page-aligned selections, one cache group, all attention layers, and non-TurboQuant K/V.

5 Environment and Method

The remote host was an Apple M5 MacBook Pro with 16 GB unified memory, macOS 26.4.1, and Metal 4. The environment used Python 3.12.7, vLLM 0.28.0+cpu, and vLLM-Metal 0.3.0.dev20260828085745 at revision 14705ad974863f68d00315655514f200366441bf. Although the package version contains +cpu, startup logs show the Metal plugin, MLX model loader, and native M5 paged-attention kernels. We therefore report both package and backend provenance.

The model was `mlx-community/Qwen3-0.6B-4bit` at revision 73e3e38d981303bc594367cd910ea6eb48349da8. The server used a 2,048-token maximum context, one sequence, 20% memory reservation, and explicit APC. It allocated 1,589.1 MB for 866 KV blocks of 16 tokens, or 13,856 tokens.

The fixed synthetic task contains one authoritative codeword and twelve distractor records. Five conditions separate no memory, a stable prefix, selected evidence, stable prefix plus selected evidence, and full context. Each condition is repeated five times contiguously. The first request is reported as cold within the running server; the remaining four form the warm mean. Streaming Server-Sent Events determine TTFT. Twenty-five requests are enough for compatibility and mechanism smoke evidence, not tail percentiles or serving claims.

6 Results

Table 1 first validates the control. Neither no-memory condition recovers the codeword, whereas selected and full context recover it in every request. Prefix plus selected memory uses 365 prompt tokens versus 1,577 for full context, a reduction of

$$1 - \frac{365}{1577} = 76.9\%.$$

Cold TTFT is 80.7 ms for prefix plus selected memory and 776.6 ms for full context. This large difference includes first-touch effects and should not be treated as a steady-state speedup. Warm TTFT narrows to 46.2 versus 52.9 ms. The engine log reports aggregate APC hit rates of 75.3% and 83.4% during the run, demonstrating that selected context did not disable prefix caching.

Table 1: vLLM-Metal smoke results. TTFT is milliseconds. “Warm cached” is not available per request in the SSE usage payload.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	875.9	42.2	n/a
Prefix	0%	333	116.6	49.4	n/a
Selected	100%	56	62.7	42.9	n/a
Prefix + selected	100%	365	80.7	46.2	n/a
Full context	100%	1577	776.6	52.9	n/a

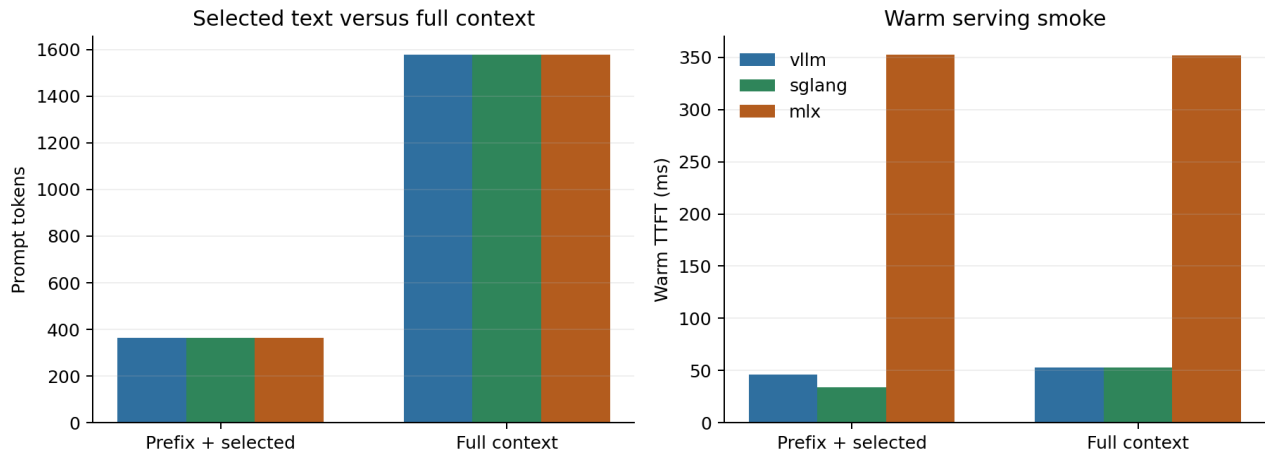


Figure 1: Shared prompt-size and warm-TTFT calibration. Timings across engines are not direct speed rankings: SGLang used an unquantized MLX conversion, whereas vLLM and MLX-LM used the 4-bit checkpoint.

Figure 1 shows the stable prompt-size reduction across all three tokenizers and the engine-specific latency scale. The comparable result is within-engine selected versus full context, not the absolute height of different-engine bars.

6.1 Native paged selected K/V

Table 2 tests the physical mechanism directly. Selected K/V occupies native vLLM-Metal pages and is consumed by the same paged-attention primitive used for sequential cache blocks. The output agrees with a dense MLX reference at every tested geometry. Warm latency changes smoothly with selected set size after compilation, and one immutable allocation is shared by all requests. These are positive block-execution and sharing results; the next experiment tests their use during generation.

6.2 Live V1 generation

Table 3 carries 32 selected native tokens through actual V1 generation while only 15 query tokens are visible. Native selected K/V and full visible context both recover the target in all five seeds. Query-only generation never recovers it, the request after native cleanup shows no leak, and substituting a wrong native memory changes the answer to that memory’s code in every seed. Mean completion time is 159.7 ms for native memory and 197.3 ms for full context; this small controlled cohort is a mechanism timing, not a throughput claim. Active selected K/V is 3.50 MiB. The experiment closes generation, causal influence, and request cleanup on this pinned Metal path.

6.3 Native PRA, APC, and concurrent V1 requests

Table 4 closes the previously separate APC and native-generation paths. vLLM reports 128–144 ordinary cached prompt tokens, and the bridge observes the same scheduler-owned boundaries before adding 32 selected tokens at the attention boundary. Thus selected pages are neither counted as APC hits nor inserted into the ordinary prefix namespace. Native recovery is 75/75 as concurrency increases from one to eight. In the strongest isolation

Table 2: Native vLLM-Metal paged attention. Each row aggregates five seeds and four concurrency levels. Cold includes first invocation for that row; warm is an immediate repeat. Sharing savings are at concurrency eight.

Selected tok.	Fraction	Cold ms	Warm ms	Max error	Shared MB saved
16	0.20%	2.769	0.489	0.003906	0.44
64	0.78%	0.602	0.523	0.0009766	1.75
256	3.12%	0.675	0.548	0.0009766	7.00
512	6.25%	0.722	0.642	0.0007324	14.00

Table 3: Five-seed live vLLM-Metal V1 generation controls. “Control success” means recovery for positive conditions, failure for disabled memory, absence of post-request leakage, or following the deliberately wrong memory.

Condition	Control success	Mean latency ms
Full visible context	100%	197.3
Selected native K/V	100%	159.7
Disabled memory (must fail)	100%	–
Post-cleanup isolation	100%	–
Wrong-memory causality	100%	–

condition, alternating rows in the same eight-request batch receive selected pages: all 20 selected rows recover the code and all 20 adjacent ordinary rows do not. A subsequent disabled batch leaks in 0/40 requests.

Figure 2 shows mean throughput increasing from 4.76 requests/s at concurrency one to 34.41 at concurrency eight. The first selected request is cold in its selection-specific APC namespace; subsequent concurrency levels reuse 128–144 visible-prefix tokens. This is offline V1 batching on one Apple M5, not an HTTP arrival-process or tail-latency benchmark. Wrong memory redirects 30/40 outputs to its counterfactual code; the imperfect rate limits the strength of that prompt-level causal control, while the earlier short-prefix experiment supplies the cleaner 5/5 result.

6.4 Matched selected text versus native K/V

The final experiment freezes 20 routed examples from each of QASPER, HotpotQA, and 2WikiMultiHopQA across five seeds. Within an example, E0 and E2 receive the same source-token sequence, question, greedy decoder, and 24-token output budget. E0 exposes the selected source as ordinary prompt text; E2 page-aligns that source, encodes it once, and attaches its native pages outside APC. Each condition runs once cold and once warm. This controls discovery: any output difference occurs after selection.

E2 reduces mean visible tokens by 90.6–93.0%. It does not yet preserve the E0 behavior: exact output parity is 5.0% on QASPER, 20.0% on HotpotQA, and 17.5% on 2Wiki. Mean E2-minus-E0 F1 is +0.008, −0.033, and −0.034, respectively. The neighboring MLX-LM and SGLang-MLX paths obtain exact parity on the same frozen selections, which rules out the cohort construction as an explanation. The remaining candidates include paged-cache precision/layout and position metadata at the vLLM-Metal boundary. Until a capture-and-replay audit matches ordinary paged K/V layer by layer, this result is a negative promotion gate, not evidence that native memory intrinsically harms QA.

Cold end-to-end completion is 12–23 ms slower for E2 after its roughly 59 ms one-time ingestion. Warm completion ranges from 6 ms faster to 13 ms slower. The offline V1 API emits the 24-token completion as one engine result, so TTFT and ITL are reported as unavailable rather than inferred from completion time.

6.5 Capture/replay and position audit

We then copy ordinary source-prefill pages from vLLM’s own paged cache and replay them through the scheduler-invisible PRA tail. Every native request uses a selection-derived APC salt, preventing query-cache reuse across audit conditions. Table 7 separates the first decoding decision from exact 24-token sequence equality.

The nominal base is decisively better than zero and preserves the first token in 5/5 cases. Captured pages improve the mean common prefix over separately encoded K/V, but do not make every generated sequence bit-identical. A second audit varies captured source length: exact sequence parity is 3/5, 1/5, 2/5, 5/5, 4/5, and 3/3 at 16, 32, 64, 128, 256, and 384 tokens. The non-monotonic pattern argues against page count as the primary

Table 4: Five-seed APC and concurrency cohort. Success denotes exact recovery for selected requests, non-recovery for ordinary/disabled requests, and exact redirection for wrong-memory controls. Throughput is the mean across seeds.

Condition	n	Success	Requests/s	APC tokens
Native PRA + APC, $C = 1$	5	5/5	4.76	0–0
Native PRA + APC, $C = 2$	10	10/10	10.33	128–144
Native PRA + APC, $C = 4$	20	20/20	19.99	128–144
Native PRA + APC, $C = 8$	40	40/40	34.41	128–144
Mixed selected / ordinary	40	40/40	35.12	144–144
Post-native disabled isolation	40	40/40	34.31	144–144
Wrong-memory redirection	40	30/40	34.51	144–144

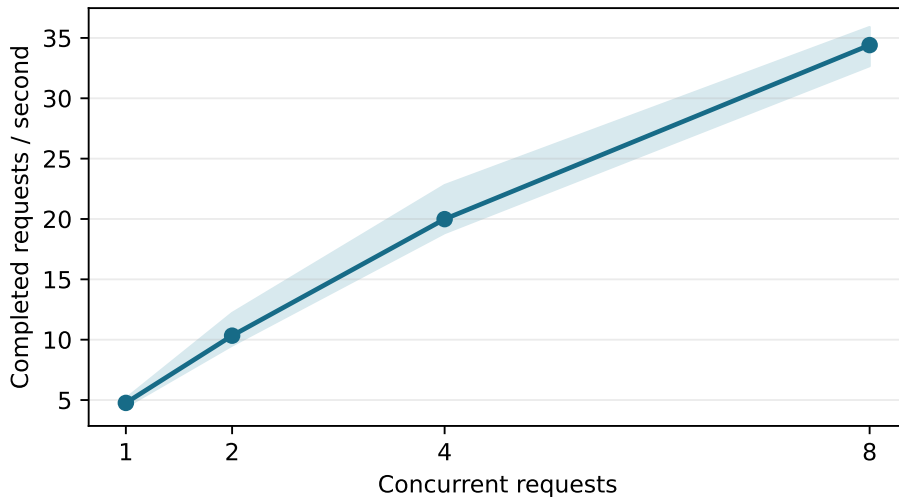


Figure 2: Live native-PRA V1 throughput with APC. The line is the five-seed mean; the band spans the observed seed range. Recovery is 100% at every concurrency shown.

defect. The remaining gap is consistent with representation and kernel numerical differences that can cascade autoregressively, but this is a diagnosis rather than proof. Logit-level capture at the fused kernel boundary remains the next correctness test.

6.6 Controlled residency pressure

The engine-neutral residency manager was also exercised on five deterministic 120-request traces with a 14 MB budget and a 36 MB working set. This is a control-plane simulation; it does not substitute for engine telemetry. Size-aware LRU reduces mean reload amplification from 0.643 to 0.397, a 38.3% relative reduction. Reuse-count and reload-cost policies reach 0.445 and 0.437. The result confirms that selected-memory lifecycle policy can matter even before learned eviction, while leaving its effect on real TTFT open.

Table 5: Cross-engine matched-selection quality. Paper 6 uses the vLLM-Metal rows; the other rows provide the common experimental reference.

Engine	Dataset	Visible E0/E2	Reduction	F1 E0/E2	Exact parity
mlx-lm	2wikimultihopqa	375/33	91.1%	0.067/0.067	100.0%
mlx-lm	hotpotqa	416/39	90.5%	0.102/0.102	100.0%
mlx-lm	qasper	399/28	93.0%	0.110/0.110	100.0%
sglang-mlx	2wikimultihopqa	375/33	91.1%	0.074/0.074	100.0%
sglang-mlx	hotpotqa	416/39	90.5%	0.099/0.099	100.0%
sglang-mlx	qasper	399/28	93.0%	0.105/0.105	100.0%
vllm-metal	2wikimultihopqa	378/33	91.2%	0.075/0.041	17.5%
vllm-metal	hotpotqa	417/39	90.6%	0.089/0.056	20.0%
vllm-metal	qasper	400/28	93.0%	0.101/0.109	5.0%

Table 6: Matched-selection latency in milliseconds. Cold completion includes one-time E2 encoding/materialization; vLLM’s offline V1 interface exposes completion but not per-token TTFT or ITL.

Engine	Dataset	Cold completion E0/E2	Warm completion E0/E2	Warm TTFT E0/E2	E2 ingest
mlx-lm	2wikimultihopqa	304.3/345.8	273.2/313.6	50.2/54.1	37.0
mlx-lm	hotpotqa	279.1/314.7	256.0/299.7	52.7/54.0	35.7
mlx-lm	qasper	303.2/350.4	270.4/311.9	46.9/51.2	35.5
sglang-mlx	2wikimultihopqa	321.6/339.4	263.9/303.3	27.9/26.7	37.1
sglang-mlx	hotpotqa	320.1/339.2	266.2/300.5	28.8/27.8	36.3
sglang-mlx	qasper	310.6/330.4	261.8/296.3	21.3/20.3	35.6
vllm-metal	2wikimultihopqa	381.0/404.2	350.1/344.2	—/—	58.9
vllm-metal	hotpotqa	386.8/407.2	339.7/352.8	—/—	59.0
vllm-metal	qasper	386.6/398.9	339.5/342.5	—/—	59.8

7 Promotion Gates

Gate	Status	Evidence
environment	closed	vLLM V1 with native Metal paged attention runs reproducibly
black-box semantics	smoke closed	selected and full context agree on the fixed task
prefix coexistence	controlled closed	128–144 APC tokens after first selection-scoped request
native block attention	controlled closed	80 Metal kernel rows; dense-reference parity
physical sharing	controlled closed	one immutable handle at concurrency 1–8
V1 metadata contract	controlled closed	selected block tables and positions remain scheduler-disjoint
live V1 generation	controlled closed	positive, disabled, wrong-memory, and cleanup controls over five seeds
concurrent isolation	controlled closed	20/20 selected recoveries and 0/20 adjacent ordinary recoveries
native APC identity	controlled closed	selection-derived salt; 75/75 selected and 0/40 disabled recoveries
matched natural QA	partial	91–93% fewer visible tokens; exact output parity only 5–20%
engine-native serving	partial	native APC and concurrency measured; connectors, online tails, and CUDA remain open
robustness	open	one model family, one kernel backend, controlled tensors

The Metal backend now accepts selected non-prefix K/V at its native paged-attention boundary and carries scheduler-invisible pages through V1 generation. Native pages now coexist with scheduler-owned APC state through eight-request V1 batches. Remaining boundaries are an online arrival process, LMCache/connectors, cross-model replication, and CUDA. The shared residency manager implements deterministic prefetch, pinning, sharing, and four eviction policies, but its pressure sweep is controlled simulation rather than vLLM scheduler telemetry.

Table 7: Five-example QASPER capture/replay diagnostic. Prefix is the mean number of generated tokens equal to E0 before the first divergence.

Native source and position	Exact	First token	Mean prefix
separate generic encoder, nominal base	2/5	5/5	15.6
captured vLLM pages, base zero	0/5	0/5	0.0
captured vLLM pages, nominal -1	1/5	4/5	9.0
captured vLLM pages, nominal base	3/5	5/5	17.8
captured vLLM pages, nominal +1	2/5	4/5	13.0

Table 8: Fixed-budget residency pressure over five seeds. Reload amplification is physical loads divided by requests; lower is better.

Policy	Mean loads	Mean evictions	Reload amplification
lru	77.2	73.8	0.643
size_aware_lru	47.6	43.6	0.397
reuse_count	53.4	49.4	0.445
reload_cost	52.4	48.4	0.437

8 Falsification and Limitations

The central thesis fails if an optimized E0/G10 integration matches an engine-aware implementation at equal quality across memory pressure, concurrency, and reuse. The present data make that falsifier stronger: the black-box baseline already preserves quality, uses 76.9% fewer visible prompt tokens than full context, and coexists with APC. A native implementation must improve resident memory, transfer, TTFT, or throughput beyond this baseline.

The current study uses a single small 4-bit model, one Apple host, synthetic answers, and five seeds. The cold requests are order-sensitive; concurrency is measured through the offline V1 engine rather than an HTTP server, and no arrival-rate, tail-latency, or HBM-pressure sweep was run. The live bridge requires page alignment, one K/V cache group, all consumer layers, and unquantized selected K/V. Apple unified memory also differs from the CUDA/LMCache environment for which many connector mechanisms were designed. Most importantly, the matched natural-QA cohort does not preserve E0 behavior on vLLM-Metal. Offline V1 also withholds per-token arrivals, so its TTFT and ITL columns remain unavailable. Selection-specific APC salts also mean that a changed PRA selection cannot reuse an APC namespace populated by a different selection, even when their visible conversation prefix is equal.

9 Related Work

vLLM introduced PagedAttention and continuous batching to reduce KV fragmentation and improve serving throughput [1]. Prefix caching reuses exact token histories; KV connectors and offload systems extend physical residency. Retrieval-augmented generation and sparse attention reduce the logical context presented to a model, but selected text is not equivalent to native non-prefix K/V. PRA combines explicit logical identity, retrieval, and native detail activation. The contribution here is the boundary between that logical mapping and vLLM’s physical mapping.

10 Conclusion

Paper 6 establishes a reproducible vLLM-Metal baseline and a control plane that cannot accidentally overclaim native execution. Selected text recovers the fixed answer with 76.9% fewer visible tokens than full context and preserves APC. That success raises, rather than lowers, the bar for deep integration. The selected-K/V path is now verified at kernel level and in live V1 generation, including wrong-memory causality and post-request isolation. Native pages also coexist with APC and preserve request isolation through concurrency eight when APC identity includes the hidden selection. PRA-vLLM is therefore an engine-native controlled prototype on Metal, but not yet a matched-quality realization. The next decisive patch must compare first-token logits at the ordinary and replayed fused-kernel boundaries, then reproduce the result through another model family and CUDA. Only after

that should online tail latency and connectors support economic claims.

References

- [1] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*, 2023.